LLPRI: Low-Latency Privacy-Preserving Vision Inference for Distributed Edge-IoT Environments

Te-Yi Kan, and Konstantinos Psounis *Fellow, IEEE*

***Abstract*—Vision inference tasks, such as object detection and instance segmentation, are essential for visual Internet of Things (IoT) applications, including human activity monitoring. However, the computational demands of high-utility vision models often exceed the capabilities of resource-constrained visual IoT devices. A solution is to offload inference to remote servers, including edge servers distributed near IoT devices. This setup, however, requires users to share visual data with remote servers, posing potential privacy risks. To address this challenge, we propose LLPRI, a Low-Latency PRivacy-preserving vision Inference system for distributed Edge-IoT environments. LLPRI comprises three key modules: a sensitive object detector, a scheduler, and an obfuscator, which jointly balance privacy, utility, and latency for the target task. Experiments on object detection and instance segmentation show that LLPRI reduces strong-adversary reconstruction SSIM by 81.1% compared with a traditional split-inference baseline, while maintaining downstream utility with $\sim$ 15% mAP50 degradation. The results also show that LLPRI leverages multi-server inference to distribute computation and reduce end-to-end inference time.**

***Index Terms*—privacy, utility, distributed systems, Internet of Things, machine learning**

I. Introduction

Visual Internet of Things (IoT) applications, such as human activity monitoring [1] and mixed reality [2], rely on vision inference tasks including object detection [3] and image segmentation [4]. Recent advances in machine learning (ML) have significantly improved the accuracy of these tasks, but these gains often rely on high-utility vision models with substantial computational demands. Even when such models can run on resource-constrained visual IoT devices, their high latency can make it difficult to meet the latency requirements of vision applications. While lightweight models can reduce local computation, they usually compromise task utility. A common solution is to offload computation to a remote cloud server, such as an AWS cloud server [5], thereby reducing the computational burden on local devices [6]. However, a single highly powerful server may not always be available or represent the most suitable deployment approach. As an alternative, distributing inference across multiple moderately powerful edge servers shared by IoT devices [7] offers several practical advantages, including exploiting parallel computation, reducing computation-related inference delay, improving scalability and resource utilization, and increasing robustness to server failures [7]–[10] In this work, we refer to this setting as a distributed Edge-IoT environment.

Regardless of whether one may use a centralized single-server cloud or, as we consider in this work, a distributed system of edge servers, offloading inference to external servers requires visual IoT devices to transmit raw visual data to them, raising significant privacy concerns because such data may contain sensitive visual information. To address this, several privacy-enhancing techniques have been explored. Homomorphic encryption (HE) [11] and multi-party computation (MPC) [12] provide strong privacy protection but usually introduce substantial computational overhead. Noise-based approaches, such as differential privacy (DP) [13], reduce privacy leakage with relatively low latency but often substantially degrade task utility. Split inference [14] avoids transmitting raw images by executing early model layers locally and sending intermediate features to the server; however, prior studies have shown that intermediate features may still be vulnerable to reconstruction attacks [15]. Adversarial representation learning (ARL) [16] can further suppress sensitive information in transmitted representations, but existing methods are often applied to the entire image, which may unnecessarily reduce utility and increase latency.

These limitations motivate a selective obfuscation approach that focuses privacy protection on sensitive regions while preserving non-sensitive regions for downstream inference. In this paper, we propose LLPRI, a low-latency privacy-preserving vision inference system for distributed Edge-IoT environments. More specifically, LLPRI targets the challenging scenario in which sensitive objects are also utility-critical and therefore cannot simply be removed or heavily distorted. It categorizes objects as sensitive or non-sensitive and applies selective obfuscation accordingly: non-sensitive objects are preserved to maintain task utility, while sensitive objects are obfuscated to reduce reconstruction-based privacy leakage. This selective design allows LLPRI to balance privacy, utility, and latency for the target vision task.

LLPRI consists of three coordinated modules: a sensitive object detector, a scheduler, and an obfuscator. The sensitive object detector identifies privacy-sensitive objects and is enhanced with privacy-aware losses and knowledge distillation to improve detection accuracy on sensitive categories. The scheduler isolates sensitive regions, partitions the input image, and assigns chunks to multiple edge servers for parallel inference. The obfuscator combines split inference with an adversarially trained mask applied to intermediate features of sensitive regions, reducing reconstruction-based privacy leakage while preserving information needed for downstream vision tasks. This modular design also enables LLPRI to support complex vision models: the sensitive object detector and scheduler operate outside the target task model, while the obfuscator only requires selecting a split point in the backbone and training a mask for the corresponding intermediate features. Therefore,

The authors are with the Department of Electrical and Computer Engineering, University of Southern California, Los Angeles, CA 90089 USA (e-mail: teyikan@usc.edu; kpsounis@usc.edu).

LLPRI does not require redesigning the full downstream model architecture. Note that while this work focuses on distributed Edge-IoT environments, this selective obfuscation framework is also applicable to centralized cloud inference.

We evaluate LLPRI in terms of utility, reconstruction-based privacy leakage, and latency, and compare it against several baselines. To demonstrate its applicability across vision tasks, we study object detection and instance segmentation, two representative tasks for visual IoT applications. For privacy evaluation, adversaries attempt to reconstruct the user's input image from the transmitted features, and we quantify reconstruction-based privacy leakage by measuring the similarity between the original and reconstructed images. Experimental results show that LLPRI reduces strong-adversary reconstruction SSIM by 81.1% compared with a traditional split-inference baseline, while maintaining downstream utility with about 15% mAP50 degradation. The results also show that LLPRI leverages multiple edge servers to distribute server-side computation and reduce end-to-end inference time in the evaluated multi-server setting. Ablation studies further analyze the impact of key design choices, including sensitive-object detection and split-point selection, on the privacy–utility–latency tradeoff. The main innovations and contributions of this work can be summarized as follows:

- We address the challenging scenario in which sensitive objects are utility-critical and therefore cannot be fully removed or heavily distorted. This differs from prior studies that assume sensitive objects do not contribute to task utility [17], [18].
- Unlike prior studies that mainly focus on pairwise tradeoffs, such as utility–latency [8] or privacy–utility [19], we jointly consider privacy, utility, and latency.
- We propose LLPRI, a modular framework that enables selective obfuscation through three coordinated modules: a sensitive object detector, a scheduler, and an obfuscator. This modular design allows LLPRI to support complex models without redesigning the full model architecture.
- We design LLPRI for distributed Edge-IoT environments, where multiple moderately powerful edge servers can be coordinated for parallel inference rather than assuming a single high-performance node [16], [20], enabling broader deployment settings.
- We develop module-level techniques that support selective obfuscation, including a sensitive object detector enhanced with privacy-aware losses and knowledge distillation, and an obfuscator that integrates split inference with adversarially trained feature masking to reduce reconstruction-based privacy leakage while preserving utility-critical information.
- We conduct extensive evaluations on object detection and instance segmentation, demonstrating that LLPRI reduces reconstruction-based privacy leakage with modest utility degradation and leverages multiple distributed servers to reduce latency. Ablation studies further analyze the impact of key design choices, including sensitive-object detection and split-point selection, on the privacy–utility–latency tradeoff.

The remainder of this paper is organized as follows. Sec.II reviews related work. Sec.III presents the problem formulation, threat model, and evaluation metrics. Sec.IV describes the design of LLPRI. Sec.V presents the experimental setup and evaluation results, while Sec.VI provides ablation studies of the key design choices. Sec.VII discusses limitations and future directions. Finally, Sec. VIII concludes the paper.

II. Related Work

Cryptography-based approaches. A common method for privacy-preserving inference is to use cryptographic algorithms, such as homomorphic encryption (HE) [11] and multi-party computation (MPC) [12]. HE enables inference directly on encrypted data, while MPC uses cryptographic schemes, such as secret sharing, to allow multiple parties to jointly compute functions on their data without revealing individual inputs to one another. Several prior works have incorporated HE and MPC into ML computations [21], [22] and IoT environments [23]. Although cryptography-based approaches are effective in preventing privacy leakage, they often introduce high computational overhead. Prior works have attempted to reduce this overhead [24], [25], but the execution time can still be prohibitive for latency-sensitive applications. Unlike cryptography-based methods, LLPRI is designed for latency-sensitive visual inference and reduces reconstruction-based privacy leakage without relying on computationally expensive encrypted computation.

Noise-based approaches. Adding noise to input data is a common lightweight method for reducing information exposure. Differential privacy (DP) [13], for example, provides strong worst-case guarantees and has been widely studied in ML workloads [26], [27] and IoT environments [28]. However, applying the noise required for meaningful DP protection often leads to substantial utility degradation [26], [27]. Moreover, the indistinguishability encouraged by DP can conflict with visual inference tasks, which require preserving instance-specific information for accurate prediction. Lighter noise-based methods without DP guarantees [29] offer lower overhead but can still substantially degrade utility because the noise is random and applied to the entire image. In contrast, LLPRI learns a downstream-utility-aware obfuscator and applies obfuscation only to sensitive regions, resulting in a better privacy–utility tradeoff.

Split inference Privacy-aware edge-assisted inference has been widely studied for IoT systems, where resource-constrained local devices seek to offload computation to nearby edge servers without directly exposing raw input data [30]. Split inference [14] is a common approach in this setting: instead of transmitting raw images, the local device executes the early layers of a model and transmits intermediate features to the edge server for downstream inference. However, these intermediate features may still contain sensitive information, allowing adversaries to infer sensitive attributes or reconstruct raw inputs [15]. To address leakage from intermediate features, prior works have proposed altering intermediate features in the contexts of mobile analytics [31] and classification [20]. Similar to these methods, LLPRI modifies intermediate features

before transmission. However, LLPRI differs by learning a downstream-utility-aware obfuscation mask and applying it selectively to sensitive regions rather than obfuscating the entire transmitted features. This selective design reduces unnecessary utility degradation and local overhead. In addition, LLPRI considers distributed Edge-IoT environments with multiple edge servers [32], whereas prior methods mainly focus on single-server inference settings.

Privacy-preserving representation learning. A major line of work seeks to reduce information exposure by transforming raw data into privacy-preserving latent representations before transmission. Adversarial representation learning (ARL) [19], [33]–[35] trains an encoder through adversarial learning to transform raw data into latent representations that suppress sensitive information locally. Although ARL-based approaches aim to balance utility and privacy, they often introduce utility degradation and additional computational overhead because they encode the entire input. Beyond ARL, prior work has also explored local transformations of inference data in edge–IoT environments [36]. LLPRI follows the same general goal of reducing information exposure before transmission, but differs by applying a learned obfuscation mask only to sensitive regions, thereby reducing unnecessary distortion and local overhead while better preserving task utility.

Privacy-preserving methods for delay-sensitive applications. Existing works have proposed privacy-preserving systems for delay-sensitive visual applications. For example, Hu *et al.* present MagicCloth [17] and Magic Rope [18] for AR applications. However, these systems require users to manually specify sensitive objects using specialized cloth or ropes. BYSTANDAR [37] reduces this burden by leveraging user eye gaze and voice cues to automatically detect and mask bystanders. Nevertheless, these systems generally aim to fully exclude sensitive objects, implicitly assuming that sensitive objects do not contribute to the target application's utility. This assumption differs from the scenario considered in this work, where sensitive objects may also be utility-critical and therefore cannot simply be removed or fully masked.

III. Preliminaries

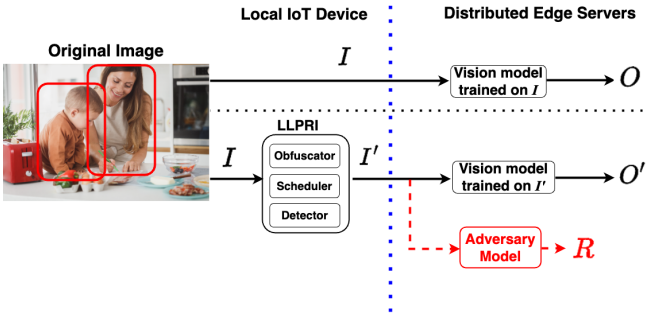


Fig. 1. LLPRI transforms the image I into an obfuscated format I'. The vision model and adversary take I' as input to produce the result O' and the reconstructed image R, respectively. The similarity between R and I reflects reconstruction-based privacy leakage, while the distance between the result O' and the original result (without obfuscation) O indicates utility degradation.

A. Problem Statement

Our goal is to design a system that enables low-latency privacy-preserving vision inference in distributed Edge-IoT environments. We consider a setting where resource-constrained visual IoT devices capture input images locally but rely on multiple edge servers to execute high-utility vision models. The system must jointly consider three key factors: privacy, utility, and latency. First, the system should reduce the exposure of sensitive visual information during inference. Local devices should avoid transmitting raw sensitive visual data directly to edge servers. Instead, sensitive information should be protected before leaving the local device. Second, the system should maintain high task utility, ensuring that supported vision tasks are performed effectively. This requires the local device to provide edge servers with sufficient task-relevant information for inference. However, such task-relevant information may also contain sensitive visual information, creating a conflict between privacy and utility. Third, the system should support low-latency inference for latency-sensitive visual IoT applications, such as human activity monitoring and mixed reality. Computation-intensive privacy-preserving techniques, such as HE and MPC, introduce substantial overhead and are often impractical in such settings. Overall, the system should reduce privacy leakage while maintaining downstream utility and low inference latency.

These requirements are considered in a distributed Edge-IoT setting with multiple edge servers, motivated by the growing adoption of edge computing paradigms [7]. The purpose of this distributed setting is to exploit parallel computation and resource availability across servers to reduce computation-related inference delay [8], [9]. This work does not focus on reducing communication overhead or communication latency; rather, the system coordinates inference across available edge servers to improve resource utilization. Note that a single powerful server can be regarded as a special case; thus, a system designed for this distributed setting can naturally accommodate single-server setups.

Since we focus on vision-based tasks, we consider sensitive information to include visual objects that users may prefer not to share with untrusted parties. In general, visual objects can be classified according to both sensitivity and task utility. Objects that are not utility-critical can be removed or heavily masked to protect privacy without compromising utility. In contrast, utility-critical objects should be handled differently depending on whether they are sensitive. Non-sensitive objects can remain unaltered to maximize performance without compromising privacy. Sensitive objects, however, must be obfuscated to preserve privacy, but any distortion applied to these objects may result in substantial utility degradation. Balancing privacy and utility for these sensitive yet utility-critical objects poses a greater challenge. This work focuses on this more challenging scenario, where sensitive objects are essential for the task and therefore affect both privacy and utility.

With the above considerations in mind, we propose LLPRI, a three-module privacy-preserving vision inference system, as shown in Fig. 1. First, the sensitive object detector identifies

sensitive objects in the input image. Second, the scheduler coordinates inference across multiple edge servers. Third, the obfuscator protects sensitive regions by applying a learned mask to intermediate features before transmission. By limiting obfuscation to sensitive regions, LLPRI preserves the utility of non-sensitive regions and reduces obfuscation delay. This design reduces reconstruction-based privacy leakage while maintaining task utility and low latency.

B. Threat Model

In this work, we consider an *honest-but-curious* scenario in which users trust only their local devices. Edge servers follow the prescribed inference protocol but may attempt to reconstruct private visual information from the data they receive. In addition, because task offloading occurs over a public network, third-party eavesdroppers may also observe the transmitted data. Therefore, adversaries may be either edge servers or eavesdroppers that attempt to reconstruct the original input image by observing the transmitted data.

LLPRI focuses on privacy-preserving inference after deployment. Privacy-preserving training is an orthogonal problem and is outside the scope of this work. As discussed in detail in Sec. IV, LLPRI comprises a latent-space embedding process, which consists of client embedding and mask generation networks, as well as a server-side vision task network. We assume that the client embedding and mask generation networks are trained before deployment in a trusted environment. During deployment, the local visual IoT device does not transmit raw input images to edge servers for inference. Instead, it transmits obfuscated intermediate features for collaborative inference. In view of the above, we consider two reconstruction adversary settings:

Weak adversary. A weak adversary lacks prior knowledge of the input image distribution and attempts to reconstruct the original image using only the transmitted obfuscated representations. This setting represents an adversary with limited auxiliary information.

Strong adversary. A strong adversary has access to image datasets with distributions similar to those of the input images. This represents a more stringent reconstruction setting, where the adversary can train a stronger reconstruction model using auxiliary data from a similar distribution [20].

C. Metrics

We evaluate the effectiveness of LLPRI by comparing its end-to-end performance against a system that performs the downstream task directly on the original images. Let I denote the original input image, and let O denote the output of the downstream model when executed on I. In contrast, LLPRI first obfuscates the image to obtain I' and then feeds I' into the downstream model, yielding output O' (see Fig. 1). The discrepancy between O and O' quantifies the utility loss introduced by obfuscation.

To assess reconstruction-based privacy leakage, we assume an adversary has access to I' and attempts to reconstruct the original visual data. The similarity between the reconstructed data R and I reflects the degree of reconstruction-based privacy leakage.

1) Privacy Metric: As outlined in Sec. III-A, we consider a reconstruction adversary that observes data transmitted from local devices and attempts to reconstruct the original images. To quantify reconstruction-based privacy leakage, we evaluate the similarity between the original images and the adversary's reconstructed images using three metrics: structural similarity index measure (SSIM) [38], semantic embedding similarity (SIM), and normalized mutual information (NMI). These metrics provide an empirical characterization of reconstruction-based privacy leakage under the considered threat model. We compute these metrics only over the sensitive regions of the images because only these regions directly affect user privacy.

SSIM. SSIM [38] is a widely used image quality assessment metric in computer vision for examining the similarity between two images. It compares attributes such as luminance and contrast to capture perceptual differences. Unlike metrics such as mean squared error (MSE), which measure pixel-by-pixel differences, SSIM is considered more effective at reflecting perceptual discrepancies as perceived by the human visual system [39]. SSIM values theoretically range from -1 to 1, although they are typically between 0 and 1, with higher scores indicating greater similarity between the original and reconstructed images. Consequently, a higher SSIM value indicates greater reconstruction-based privacy leakage, as it implies that the adversary has reconstructed an image more similar to the original; a perfect reconstruction would yield SSIM $= 1$.

SIM. The second metric, SIM, evaluates the similarity between the semantic embeddings of the original image set I and the reconstructed image set R. Specifically, we first employ a pre-trained vision-based embedding model, such as ResNet [40] or VGG [41], to generate semantic embeddings for both the original and reconstructed images. We then calculate the cosine similarity (CS) between the semantic embeddings of the paired images. Formally, suppose that the embeddings of image instances $i \in I$ and $r \in R$, generated by the pre-trained vision-based embedding model, are represented as $Emb(i)$ and $Emb(r)$, respectively. The SIM between I and R can be written as:

$$SIM(I, R) = \mathbb{E}_{i,r}\left[CS(Emb(i), Emb(r))\right], \tag{1}$$

where $CS(x_1, x_2) = x_1 \cdot x_2 / (\max(\|x_1\|_2, \epsilon) \cdot \max(\|x_2\|_2, \epsilon))$ is the cosine similarity between two vectors. Here, ϵ is a small constant used to avoid division by zero. The range of SIM is the same as that of CS, which is $[-1, 1]$. Similar to SSIM, a higher SIM value indicates greater reconstruction-based privacy leakage, as it means that the reconstructed image is more semantically similar to the original image. A value of $SIM = 1$ indicates maximum semantic similarity under the selected embedding model.

NMI. The third metric quantifies reconstruction-based privacy leakage using mutual information (MI) between an original image and its reconstruction. MI measures the statistical dependence between two variables and formally describes how much knowledge of one variable reduces uncertainty about the other [42]. Because of this information-theoretic interpretation, MI is commonly used to characterize information leakage and has also been adopted for image similarity assessment [43].

Let I denote the set of original images and R the set of reconstructed images produced by the adversary. We define our MI-based privacy metric as:

$$NMI(I,R) = \mathbb{E}_{i,r}\left[MI(i,r)/H(i)\right], \tag{2}$$

where $i \in I$ and $r \in R$ are corresponding image instances. Here, $MI(i,r)$ measures how much information the reconstruction r reveals about the original image i, while $H(i)$ denotes the entropy of i. This asymmetric normalization directly captures the fraction of the original image's information that is recoverable from the reconstruction, with higher values indicating greater reconstruction-based privacy leakage.

2) Utility Metric: Generally, obfuscation systems preserve users' privacy at the cost of utility. We therefore measure the utility degradation introduced by obfuscation as follows. Suppose that the task outputs with and without obfuscation are O' and O, respectively, and that their corresponding utilities are $U(O')$ and $U(O)$. We define the normalized utility degradation as:

$$U_{\text{deg}}(O,O') = (U(O) - U(O'))/U(O). \tag{3}$$

The utility function $U(\cdot)$ is task-specific. For example, when object detection is the target task, $U(\cdot)$ represents mean average precision (mAP) [44].

IV. Methodology

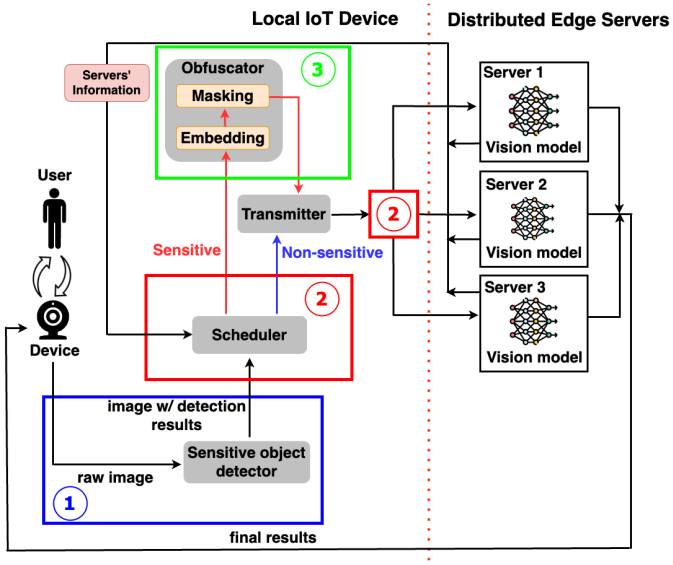


Fig. 2. System details of LLPRI.

Fig.2 presents the complete framework of LLPRI, which consists of three main modules: 1) a sensitive object detector (Sec.IV-A), 2) a scheduler (Sec.IV-B), and 3) an obfuscator (Sec.IV-C). At a high level, LLPRI works as follows. First, the sensitive object detector identifies sensitive objects in the input image. Next, the scheduler isolates these objects from the rest of the image. Based on the resource status of the edge servers, the scheduler further divides the image into chunks and assigns them to different servers. Image chunks without sensitive content are sent directly to the transmitter, which forwards them to the designated edge servers, while chunks containing sensitive objects are routed to the obfuscator. The obfuscator executes the local layers up to the selected split point and applies a learned mask to the resulting intermediate features of sensitive regions to reduce reconstruction-based privacy leakage. Once obfuscation is complete, these chunks are also sent to the transmitter for delivery to the servers.

LLPRI is designed as a modular framework rather than a modification of a specific downstream model. The sensitive object detector and scheduler operate outside the target vision model, while the obfuscator only requires selecting a split point in the target model, executing the local layers up to that split point, and applying a learned mask to the corresponding intermediate features. This design allows LLPRI to support different vision models without redesigning the full model architecture.

A. Sensitive Object Detector

We envision that users can provide a list of sensitive object categories to guide the training of a customized detector, which is then deployed locally to identify sensitive objects in input images. Because local devices have limited memory and storage, the detector must be both fast and compact. Accordingly, we choose YOLOv8n [45] as the basis for our sensitive object detector because it provides a strong balance between inference speed and detection performance. In addition, YOLOv8n offers strong community support, such as the Ultralytics API [45], and a straightforward training pipeline, which reduces engineering overhead and enables rapid iteration and deployment. Additional analysis is provided in Sec. VI-A.

However, using a lightweight model presents privacy-related challenges: detection performance may suffer, leading to class prediction errors, bounding-box shifts, and missed sensitive objects, which may increase the risk of privacy leakage. False detections of non-sensitive objects are less critical from a privacy perspective, but they may still trigger unnecessary obfuscation and reduce task utility. To improve detector reliability, we modify the YOLOv8n loss function and employ knowledge distillation [46] during training, with particular emphasis on sensitive-object recall and localization accuracy.

A simple way to improve recall is to lower the detection confidence threshold for sensitive classes, but this increases the number of regions that must be obfuscated and therefore raises local computation cost. Instead, we incorporate two privacy-oriented loss terms, the privacy-enhanced false-negative loss $\mathcal{L}_{\text{FN}}$ and the privacy-enhanced false-positive loss $\mathcal{L}_{\text{FP}}$, into the original YOLO loss function [45]. These terms bias training toward high recall on sensitive classes while controlling unnecessary false positives. The privacy-enhanced false-negative loss is defined as:

$$\mathcal{L}_{\text{FN}} = \frac{1}{|\mathcal{I}_{\text{sens}}|} \sum_{i \in \mathcal{I}_{\text{sens}}} \sum_{a \in \mathcal{A}^i_{\text{sens}}} \text{BCE}(p_a, t_a), \tag{4}$$

where $\mathcal{I}_{\text{sens}}$ denotes the set of images containing at least one sensitive object, and $\mathcal{A}^i_{\text{sens}}$ denotes the set of assigned

predictions for sensitive objects in image i. The BCE term is the binary cross-entropy between the predicted sensitive-class probability p_a for prediction a and its corresponding soft target score t_a. Following YOLOv8's task-aligned assignment, t_a reflects how strongly prediction a matches a sensitive ground-truth object based on classification and localization alignment. By encouraging assigned predictions to produce high confidence for sensitive classes, $\mathcal{L}_{\mathrm{FN}}$ reduces the likelihood of missing sensitive objects.

Complementarily, the privacy-enhanced false-positive loss is defined as:

$$\mathcal{L}_{\mathrm{FP}} = \frac{1}{|\mathcal{I}_{\mathrm{nonsens}}|} \sum_{i \in \mathcal{I}_{\mathrm{nonsens}}} \sum_{a \in \mathcal{A}^i} \mathrm{BCE}(p_a, 0), \tag{5}$$

where $\mathcal{I}_{\mathrm{nonsens}}$ denotes the set of images without sensitive objects, and $\mathcal{A}^i$ denotes the set of predictions in image i. For these images, $\mathrm{BCE}(p_a, 0)$ penalizes spurious sensitive-class predictions that would trigger unnecessary obfuscation. We apply this term only to images without sensitive ground truth to avoid conflicting with the false-negative objective on images that contain sensitive objects.

With these two additional loss terms, the overall detection loss becomes:

$$\mathcal{L} = \gamma_{\mathrm{ori}} \cdot \mathcal{L}_{\mathrm{ori}} + \gamma_{\mathrm{FN}} \cdot \mathcal{L}_{\mathrm{FN}} + \gamma_{\mathrm{FP}} \cdot \mathcal{L}_{\mathrm{FP}}, \tag{6}$$

where $\mathcal{L}_{\mathrm{ori}}$ is the original YOLOv8 loss [45], and γ_{ori}, γ_{FN}, and γ_{FP} control the contribution of each term. We choose $\gamma_{\mathrm{FN}} > \gamma_{\mathrm{FP}}$ to explicitly prioritize recall over precision for sensitive classes, reducing the chance that sensitive regions remain unobfuscated during inference.

In addition to missed detections, inaccurate bounding-box localization can also cause privacy leakage, as portions of a sensitive object may fall outside the predicted box and remain unobfuscated. To further improve sensitive-object classification and localization, we incorporate response-based knowledge distillation [46]. Specifically, we first train a larger YOLOv8x model using the revised loss function in (6) and treat it as the teacher network. The lightweight YOLOv8n detector is then trained to mimic the teacher's outputs through the following distillation loss:

$$\mathcal{L}_{\mathrm{KD}} = \frac{1}{N} \sum_{i=1}^{N} w_i \left[\gamma_{\mathrm{cls}} \mathrm{BCE}(C_i^{\mathrm{t}}, C_i^{\mathrm{s}}) + \gamma_{\mathrm{bbox}} \mathrm{KL}(B_i^{\mathrm{t}}, B_i^{\mathrm{s}}) \right], \tag{7}$$

where N is the number of teacher-student prediction pairs used for distillation. C_i^{t} and C_i^{s} denote the teacher and student classification outputs for the i-th prediction, respectively. Similarly, B_i^{t} and B_i^{s} denote the corresponding teacher and student bounding-box localization outputs, represented in YOLOv8's distributional box format rather than as decoded box coordinates [45]. The BCE and Kullback–Leibler (KL) divergence terms measure the classification and localization distillation losses, respectively, while γ_{cls} and γ_{bbox} control their relative weights.

We further apply class-dependent weights w_i, assigning $w_i = w_{\mathrm{sens}}$ for sensitive-class predictions and $w_i = w_{\mathrm{nonsens}}$ for non-sensitive predictions, with $w_{\mathrm{sens}} > w_{\mathrm{nonsens}}$. This design places greater emphasis on sensitive classes during distillation, allowing the student model to inherit stronger classification and localization cues for sensitive objects.

B. Scheduler

As mentioned in Sec. III-A, LLPRI is designed for distributed Edge-IoT environments with multiple edge servers, as in [8], [9]. Therefore, we incorporate a scheduler into LLPRI to coordinate image partitioning and task assignment across available servers. Given an input image and the sensitive-object detection results, the scheduler separates sensitive and non-sensitive regions, routes sensitive regions through the obfuscator, and assigns the resulting image chunks to edge servers based on estimated resource availability. This allows LLPRI to exploit distributed computation while performing selective obfuscation.

Consider M edge servers, where each server m has computing capacity f_m and evenly shares its capacity among the tasks running on it. We approximate computational demand using input image size, assuming that the resources required for a vision task scale linearly with the input size. This assumption provides a practical approximation for scheduling and can be generalized to more complex forms. In practice, the relationship between input size and resource demand can be estimated by profiling vision tasks.

Given an image with detection results from the sensitive object detector, the scheduler first extracts each predicted bounding box to create an individual sensitive crop, such that each crop contains one sensitive object. If two bounding boxes overlap, the overlapping area is preserved in both sensitive crops to ensure that each sensitive object remains complete within its corresponding crop. This conservative design helps maintain downstream utility by avoiding incomplete object crops. The tradeoff is that overlapping regions may be processed more than once, introducing redundant computation. However, this redundancy can be reduced by merging highly overlapping sensitive regions before scheduling. The sensitive crops are then handled by the sensitive-region scheduling mechanism, while the remaining non-sensitive regions are handled by the non-sensitive-region scheduling mechanism.

sensitive-region scheduling. Upon receiving a batch of sensitive image crops, the scheduler sorts them in descending order of estimated computational demand, prioritizing crops with higher demand. It then estimates the inference time for each crop n on each server m as:

$$t_n^{m,\mathrm{est}} = F_n / (f_m / (N_m^{\mathrm{est}} + 1)), \tag{8}$$

where F_n denotes the estimated computational demand of crop n, and N_m^{est} is the estimated number of tasks running on server m, obtained through the scheduler's periodic probes. The crop is then assigned to the server with the lowest $t_n^{m,\mathrm{est}}$.

Non-sensitive-region scheduling. Consider a non-sensitive image region of size L. The scheduler partitions this region across a selected set of edge servers $M_{\mathrm{sel}} \subseteq M$ in proportion to their estimated available computing capacity. The size of the portion assigned to server $m \in M_{\mathrm{sel}}$ is:

$$L_m = L \cdot (f_m / (N_m^{\mathrm{est}} + 1)) / \textstyle\sum_{m' \in M_{\mathrm{sel}}} (f_{m'} / (N_{m'}^{\mathrm{est}} + 1)), \tag{9}$$

where servers with more available computing capacity receive larger image portions, while more heavily loaded servers receive smaller portions. Under the assumption that computational demand scales approximately with image size, this strategy balances the estimated processing time across servers. To avoid excessively small partitions when many servers are available, the scheduler imposes a minimum partition size and uses only the selected subset of servers for scheduling.

Overall, the scheduler is designed to coordinate selective obfuscation and distributed inference in Edge-IoT environments. It uses input size as a lightweight proxy for computational demand to support low-overhead online scheduling, rather than attempting to solve a universally optimal scheduling problem. Because LLPRI is modular, the scheduler can be extended with more advanced profiling-based, communication-aware, or learning-based strategies without modifying the sensitive object detector or obfuscator.

C. Obfuscator

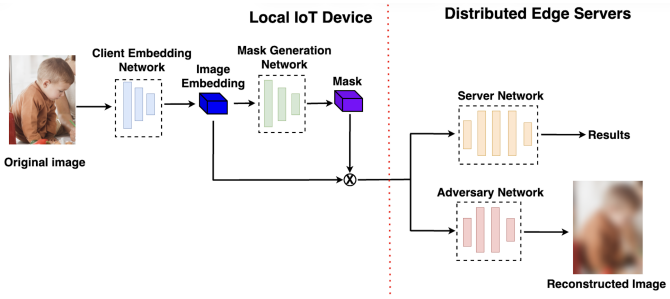


Fig. 3. Obfuscator architecture

The main objective of the obfuscator is to transform sensitive image regions into high-dimensional latent embeddings before offloading, thereby reducing direct exposure of raw sensitive visual data. A possible approach is to introduce an additional image encoder to map the input image into such latent embeddings [34]. However, this would add extra computation on the local device and increase end-to-end latency. To avoid this overhead, LLPRI leverages split inference [14], [35]. Specifically, the target vision model is divided at a selected split point into two parts: the "client embedding network," which corresponds to the front portion of the original target model and runs on the local device, and the "server network," which corresponds to the remaining portion and runs on the edge server. The client embedding and server networks together form the original target vision model. Therefore, the client embedding network reuses the front portion of the target model to generate intermediate features, which serve as the latent embeddings transmitted for split inference, without introducing an additional encoder. The server network then completes downstream task inference.

Transforming sensitive image regions into intermediate features reduces direct exposure of raw visual data, but it does not eliminate reconstruction-based privacy leakage. Prior studies have shown that intermediate features can still be vulnerable to reconstruction attacks [31], [47]. Therefore, LLPRI incorporates a mask generation network to selectively mask features before transmission. Although the mask generation network introduces slight additional computation, it reduces reconstruction-based privacy leakage by suppressing features that are useful for image reconstruction while preserving features needed for the downstream vision task.

Fig.3 illustrates the obfuscator, which includes the client embedding network and the mask generation network. The client embedding network takes a sensitive image region as input and produces intermediate features at the selected split point of the target vision model. The selected split point determines the boundary between the client embedding network and the server network and affects the privacy–utility–latency tradeoff; we evaluate this effect through split-point ablation studies in Sec.VI-B. The intermediate features are then passed to the mask generation network, which produces a corresponding mask. The obfuscator applies this mask to the intermediate features through element-wise multiplication: activations that are more likely to support adversarial reconstruction of sensitive visual details and less relevant to the downstream task are suppressed, while task-critical activations are preserved. The masked features are then offloaded to the edge servers, where the server network completes downstream inference and generates the final task output. Under our threat model, an adversary may intercept the transmitted masked features and attempt to reconstruct the sensitive image region.

Training. During training, we introduce a proxy reconstruction adversary and adopt adversarial training [48] to train the client embedding network, server network, and mask generation network. The proxy adversary attempts to reconstruct the original sensitive image region from the transmitted masked features, while the obfuscator is trained to make such reconstruction more difficult without substantially degrading downstream task utility. Specifically, let θ_c, θ_m, θ_s, and θ_a denote the parameters of the client embedding network, mask generation network, server network, and proxy reconstruction adversary, respectively. We optimize these components alternately during training, as described below. The client embedding and server networks correspond to the split target vision model and are optimized to minimize the downstream utility loss:

$$\min_{\theta_c,\theta_s} \mathcal{L}_{\text{utility}}(O, O^{\text{GT}}), \tag{10}$$

where O and O^{GT} denote the prediction and ground truth of the server network, respectively. The utility loss function $\mathcal{L}_{\text{utility}}$ is defined as in the original, non-split vision network. The proxy reconstruction adversary is trained to reconstruct the original sensitive image region from the transmitted masked features by minimizing the reconstruction loss:

$$\min_{\theta_a} \mathcal{L}_{\text{privacy}}(I, R), \tag{11}$$

where I and R denote the original and the reconstructed images, respectively. The loss $\mathcal{L}_{\text{privacy}}$ measures the difference between I and R, and lower values indicate more successful reconstruction by the adversary. The mask generation network is trained to balance downstream utility and reconstruction

resistance. Specifically, it minimizes the utility loss while maximizing the reconstruction loss of the proxy adversary:

$$\min_{\theta_m} \left[\mathcal{L}_{\text{utility}}(O, O^{\text{GT}}) - \lambda \mathcal{L}_{\text{privacy}}(I, R) \right], \tag{12}$$

where λ controls the tradeoff between downstream task utility and reconstruction-based privacy leakage.

V. Evaluation

A. Implementation Details

We implement LLPRI and all baselines using PyTorch 2.4.1 with CUDA 12.1. Experiments are conducted on two machines: a server equipped with a workstation-grade NVIDIA RTX A6000 GPU and a desktop equipped with a consumer-grade NVIDIA RTX 3070 GPU. The desktop is used as the local execution platform for LLPRI, while the server is used as the remote edge server and runs the server network, corresponding to the remaining layers of the split target vision model. We train and evaluate the system on the COCO dataset [44]. In our setup, people, dogs, and cats are treated as sensitive objects.

We evaluate LLPRI on object detection and instance segmentation, two representative vision tasks for latency-sensitive visual IoT applications [3], [49], such as human activity monitoring. Specifically, we first apply LLPRI to object detection using Faster R-CNN [50] as the target vision model and then extend the evaluation to instance segmentation using Mask R-CNN [51]. Since instance segmentation is more complex than object detection [49] and Mask R-CNN includes an additional mask prediction head, this extension demonstrates the applicability of LLPRI across vision tasks with different levels of complexity.

As detailed in Sec.IV-C, we split the target vision model within the backbone. For both tasks, we utilize ResNet-18 [40] as the backbone, with the split placed after Residual Layer 1. We choose Residual Layer 1 as the split point for two reasons. First, this configuration keeps the on-device computation and memory demand low; later residual blocks introduce higher FLOPs and memory traffic, which would increase latency on resource-constrained local devices. Second, early-layer activations preserve more spatial and textural information, making them more vulnerable to reconstruction attacks and therefore more challenging to protect. Evaluation under this challenging setting provides a meaningful assessment of the effectiveness of our masking module. If the method performs well at an early split point, where reconstruction risk is relatively high, it is also expected to remain effective at later split points, where features tend to be more abstract and less vulnerable to reconstruction. Further comparisons and tradeoff analyses across different split points are provided in Sec.VI-B. For the mask-generation network, we adopt a U-Net-based architecture [52]. To simulate a reconstruction adversary, we implement a U-Net-based architecture to reconstruct the original sensitive image regions from the transmitted embeddings. During training, we define the privacy loss in (11) as $\mathcal{L}_{\text{privacy}}(I, R) = 1 - \text{SSIM}(I, R)$.

B. Baselines

We evaluate LLPRI in terms of reconstruction-based privacy leakage, downstream-task utility, and latency, and compare it against four baselines:

Unaltered Feature: The unaltered feature baseline is a conventional split-inference approach in which the local device directly transmits the output feature maps of the client embedding network to edge servers without any modification.

Gaussian Noise: The Gaussian noise baseline adds noise sampled from a Gaussian distribution to the original input image before offloading, thereby reducing the direct exposure of raw visual information.

Random Mask: The random mask baseline follows the same mechanism as the obfuscator in LLPRI. However, instead of using a learned mask generation network, it randomly generates a mask to remove activations from the output feature maps of the client embedding network.

PCA Embedding: The PCA embedding baseline uses principal component analysis (PCA) to generate an embedding of the original image, which is then transmitted to edge servers instead of the raw image.

TABLE I
Utility degradation percentage

Obfuscator	Utility degradation (mAP50)	
	Detection	Segmentation
LLPRI	14.99%	14.32%
Unaltered Feature	0%	0%
Gaussian Noise	71.79%	71.85%
Random Mask	47.13%	48.80%
PCA Embedding	73.63%	76.04%

Note: Utility degradation is reported for object detection and instance segmentation. We use mAP50 (mean average precision at an IoU threshold of 50%) as the utility metric.

C. Utility

To better assess utility degradation under comparable reconstruction-based privacy levels, we configure the baselines to achieve privacy levels similar to those of LLPRI. Specifically, based on the privacy–utility tradeoffs shown in Figs. 4a and 4e, we select baseline parameters that provide slightly lower privacy levels than LLPRI. This gives the baselines a modest advantage in utility, since weaker obfuscation generally leads to lower utility degradation.[1] Note that the unaltered feature baseline is not configurable. The exact configurations are as follows: the activation removal percentage for the random mask baseline is set to $\alpha = 50\%$, the variance for the Gaussian noise baseline is set to $\sigma = 10$, and the retained spectrum percentage for the PCA embedding baseline is set to $\beta = 65\%$. The resulting utility degradation under these configurations is summarized in Table I.

First, the unaltered feature baseline introduces no utility degradation because it simply splits the trained target model without modifying the intermediate feature maps, thereby preserving the same output as the original model. However, as discussed in the next two sections, this comes at the cost

[1]This configuration provides a conservative comparison for LLPRI while avoiding extensive tuning to match the same privacy levels across all methods.

TABLE II
PRIVACY LEAKAGE UNDER A STRONG AND WEAK ADVERSARY

Obfuscator	Strong Adversary				Weak Adversary			
	SSIM	SIM		NMI	SSIM	SIM		NMI
		ResNet50	VGG16			ResNet50	VGG16	
LLPRI	0.1490	0.4973	0.1269	0.1010	0.0972	0.4491	0.1003	0.1190
Unaltered Feature	0.7879	0.8636	0.8059	0.5093	0.5897	0.5382	0.2113	0.3818
Gaussian Noise	0.6258	0.7328	0.5790	0.4307	0.3784	0.4768	0.1618	0.3321
Random Mask	0.2743	0.5277	0.1943	0.2440	0.2853	0.4901	0.1299	0.2211
PCA Embedding	0.5191	0.6600	0.4376	0.3791	0.3828	0.5636	0.2610	0.3277

Note: We report results for both strong and weak adversaries. For all privacy metrics, smaller values indicate stronger privacy protection.

of limited resistance to reconstruction attacks. In contrast, the other baselines reduce utility because they obfuscate either the input image or the intermediate features.

Next, LLPRI achieves significantly lower utility degradation on both target tasks than the other three baselines. Two factors contribute to this result: 1) with the help of adversarial training, LLPRI selectively suppresses activations that could help an adversary reconstruct sensitive visual details while preserving or enhancing those that contribute to the target task; and 2) LLPRI applies obfuscation only to sensitive regions, leaving non-sensitive regions unaltered.

Additionally, the Gaussian noise and random mask baselines incur more than $3\times$ the utility degradation of LLPRI across both target tasks. In the Gaussian noise baseline, noise is generated randomly, while in the random mask baseline, activations are removed arbitrarily. This randomness may disrupt task-relevant information because each obfuscation instance involves different noise values or activation-removal patterns, even for the same input. Consequently, the downstream model produces less consistent and less accurate predictions.

Finally, the PCA embedding baseline results in the greatest utility degradation. This is likely because retaining only 65% of the spectrum to achieve the desired privacy level removes substantial visual information, thereby reducing downstream-task utility.

D. Privacy

Similarly, we select baseline parameters that result in slightly lower utility than LLPRI. Since lower utility generally corresponds to stronger obfuscation, this setting gives the baselines a modest advantage in reconstruction-based privacy protection. The selected parameter values are $\alpha = 30\%$, $\beta = 95\%$, and $\sigma = 1$; see Figs. 4a and 4e. For this evaluation, we utilize two common vision embedding models, ResNet-50 and VGG16, as image encoders for computing SIM. Note that these encoders are used only to compute the SIM privacy metric and are not the target vision model backbones used by LLPRI. Moreover, we examine both strong and weak reconstruction adversaries. Finally, we assume that the adversary can intercept all data transmitted from the local device, which provides a conservative evaluation setting. If an adversary instead compromises only a subset of the edge servers, it may observe only part of the transmitted data, resulting in lower reconstruction-based privacy leakage than that reported here.

Under the strong-adversary setting, the unaltered feature baseline exhibits the highest privacy leakage. This is expected because it transmits unmodified intermediate features without obfuscation. Although the raw image is not shared, the features still retain sufficient visual information for the adversary to reconstruct the original sensitive regions. In contrast, LLPRI achieves the lowest privacy leakage among all evaluated approaches. This result is attributable to the adversarial training process, which suppresses activations that could help an adversary reconstruct sensitive visual details.

Interestingly, the evaluated approaches generally exhibit higher SIM values when ResNet-50 is used as the SIM encoder than when VGG16 is used. For LLPRI, the unaltered feature, and random mask baselines, the client-side network is ResNet-based. Consequently, their transmitted features may preserve semantic information that is more readily captured by ResNet-based embeddings. By contrast, the PCA embedding and Gaussian noise baselines distort the input image rather than directly modifying intermediate semantic features. ResNet-50, because of its depth and residual architecture, may capture semantic information more effectively than VGG16 [53], resulting in higher SIM values for these baselines as well.

As shown in Table II, the weak adversary generally produces lower-quality reconstructions than the strong adversary. This is expected because the weak adversary lacks access to image datasets with distributions similar to those of the user input images. However, in the low-NMI regime, the weak adversary occasionally produces slightly higher NMI values than the strong adversary, such as 0.1190 versus 0.1010 for LLPRI. One possible explanation is that the adversary is optimized using an SSIM-based loss rather than NMI. Therefore, improvements in perceptual reconstruction quality do not necessarily lead to monotonic changes in NMI, particularly when the measured NMI values are small. Differences in reconstruction smoothness, intensity distributions, and the sensitivity of the MI estimator may also contribute to this behavior. Overall, these observations indicate that NMI may exhibit greater variability in the low-value regime. Nevertheless, across the evaluated metrics and adversary settings, LLPRI generally achieves the lowest privacy leakage.

E. Tradeoff between Utility and Privacy

We compare utility and reconstruction-based privacy leakage under various parameter settings. Specifically, the variance σ of the Gaussian noise baseline ranges from 1 to 15, the activation removal percentage α of the random mask baseline varies from 10% to 70%, and the retained spectrum percentage β of the PCA embedding baseline varies from 65% to 95%. We evaluate reconstruction-based privacy leakage under the

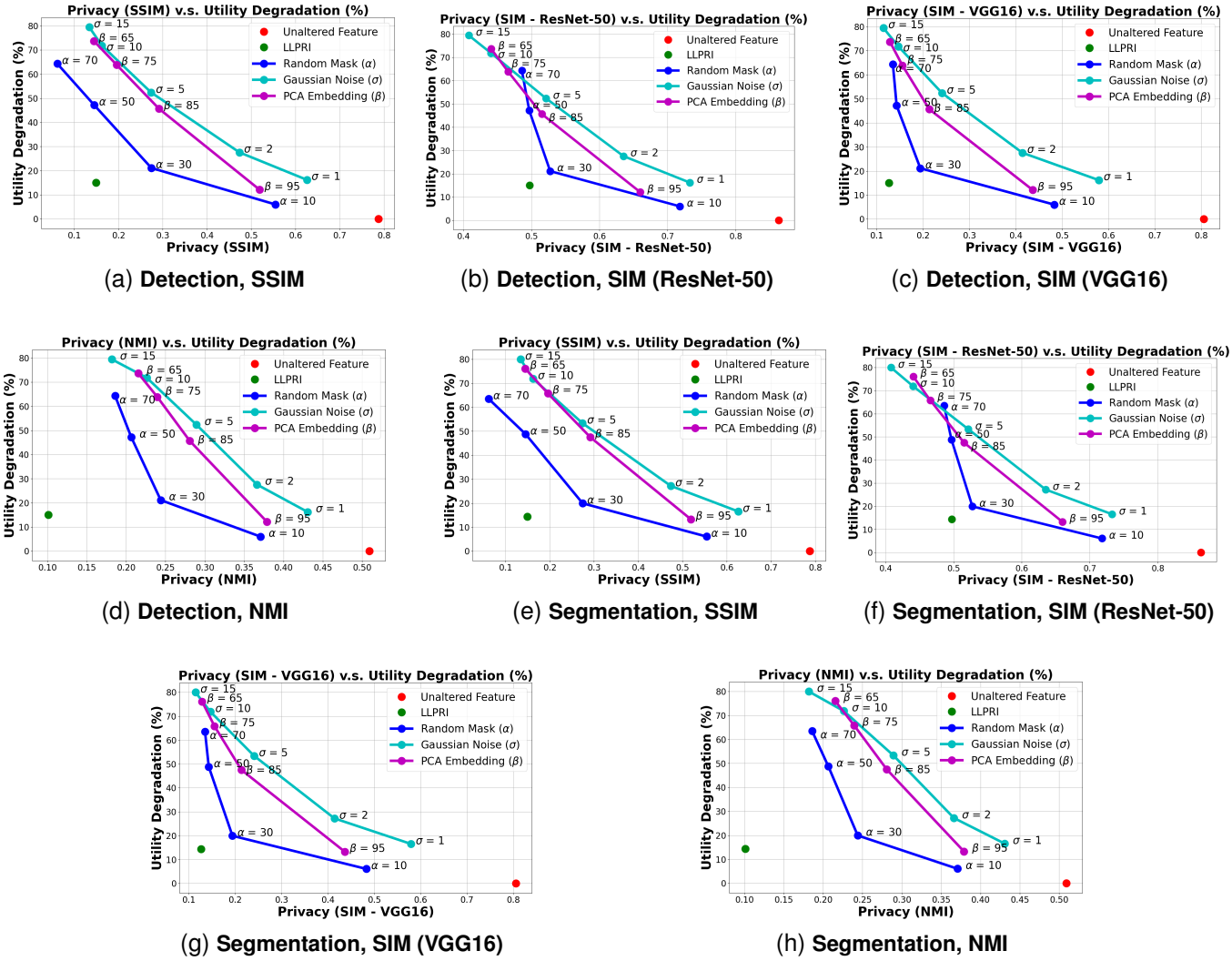


Fig. 4. Privacy–utility tradeoff results. Note that the reconstruction-based privacy leakage (lower values indicate better privacy) is measured using SSIM, SIM with ResNet-50 and VGG16 as the image encoders, and NMI. We evaluate the tradeoff separately for object detection and instance segmentation, using mAP50 as the utility metric. Utility degradation is reported as the mAP50 degradation percentage.

strong-adversary setting, which provides a more stringent assessment of resistance to reconstruction attacks. The results are presented in Fig. 4, with the X-axis representing reconstruction similarity, where larger values indicate greater privacy leakage, and the Y-axis representing utility degradation.

First, we find that, for both target tasks and across all privacy metrics, LLPRI is positioned near the bottom-left corner, indicating that it achieves low utility degradation while maintaining low privacy leakage. In contrast, the unaltered feature baseline appears below LLPRI but is located at the far right, meaning that it provides the highest utility but leaks the most private visual information.

Next, we observe that the other three baselines exhibit clear tradeoffs between utility and privacy. The Gaussian noise baseline shows that utility decreases as the noise variance increases, while privacy improves. This is expected because a higher variance introduces stronger obfuscation noise. The PCA embedding baseline also demonstrates a privacy–utility tradeoff: as more of the spectrum is retained, more image information is preserved, resulting in higher utility but greater privacy leakage. Generally, retaining more than 90% of the spectrum may be necessary to achieve favorable utility [54]. This behavior is also observed in Figs. 4a–4h, where utility degradation is approximately 13% when 95% of the spectrum is retained. Similarly, for the random mask baseline, when fewer activations are removed, more information is preserved, resulting in higher utility but greater privacy leakage. Figs. 4a–4h show that the random mask baseline achieves lower utility degradation than LLPRI when $\alpha = 30\%$. However, under these settings, it also exhibits substantially greater privacy leakage than LLPRI.

Finally, among the three baselines, the random mask baseline exhibits a more favorable privacy–utility tradeoff than the Gaussian noise and PCA embedding baselines. This is likely because the random mask baseline first transforms the image into intermediate features and then removes a subset of activations from those features. However, because its mask is generated randomly rather than learned according to the

downstream-task and reconstruction objectives, its tradeoff remains less favorable than that of LLPRI.

To further illustrate the challenge of obtaining meaningful DP guarantees without severe utility degradation, we derive the differential-privacy bound corresponding to the Gaussian noise baseline. We model the baseline as a Gaussian mechanism $M(x) = f(x) + \mathcal{N}(0, \sigma^2 I_m)$, where f(x) is the clipped and normalized image of the sensitive object and m is the number of pixels in the image. In our dataset, a sensitive object contains $m \approx 7800$ pixels on average. Under a neighboring-input definition in which two normalized images may differ arbitrarily, the ℓ_2 sensitivity is upper-bounded by $\Delta = \sqrt{m}$. Using this sensitivity and $\delta = 10^{-6}$, the Gaussian mechanism guarantees (ε, δ)-DP with $\varepsilon = (\Delta/\sigma) \cdot \sqrt{2\ln(1.25/\delta)}$ [55].

Using this formulation, when the noise standard deviation matches the range of the normalized input, i.e., $\sigma = 1$ for images scaled to $[0, 1]$, the resulting privacy bound is $\varepsilon = 467.987$, which does not provide a meaningful privacy guarantee. Increasing the noise standard deviation by an order of magnitude to $\sigma = 10$ reduces this value to $\varepsilon = 46.80$, which remains large. Moreover, this level of noise already causes more than 70% utility degradation in our task. Achieving a meaningful DP guarantee ($\varepsilon < 1$) would require $\sigma > 467.98$, which would overwhelm the normalized image signal and severely degrade utility. These results illustrate that directly applying a pixel-level Gaussian mechanism leads to an unfavorable privacy–utility tradeoff for the evaluated visual inference tasks.

F. Latency

In this subsection, we first evaluate the end-to-end task duration, which consists of two components: the obfuscation duration, defined as the time required to execute the obfuscation approach on the local device, and the target-model duration, defined as the time required to execute the server-side portion of the target vision model on the edge server. We focus on computation latency in this evaluation and do not include network transmission latency; communication-aware scheduling can be incorporated into LLPRI as discussed in Sec.IV-B. All evaluated approaches are experimentally assessed using a single edge server. Note that, for LLPRI in the single-server setting, task scheduling across multiple servers is unnecessary and is therefore disabled. The results are summarized in Table III(a).

We also evaluate LLPRI in a multi-server scenario to assess its practicality when multiple edge servers are available. In this setting, LLPRI leverages its scheduler to distribute computation across the servers. The multi-server evaluation is conducted through a simulation with three servers and three clients running LLPRI, assuming that each server has the same configuration as the server used in the single-server setup. The workload is configured to maintain a comparable load per server across the two settings. Thus, the observed latency differences primarily result from increased parallelism rather than differences in server capacity or workload. The results are presented in Table III(c).

Table III(a) shows that LLPRI incurs slightly longer obfuscation duration than most baselines, except for the PCA embedding baseline. This additional overhead primarily stems from the sensitive object detector, as detailed in Table III(b). Nonetheless, this overhead is the cost of enabling selective obfuscation: the detector identifies sensitive objects so that obfuscation can be applied only to sensitive regions while leaving non-sensitive regions unaltered. This design helps preserve downstream utility while reducing reconstruction-based privacy leakage. In contrast, the Gaussian noise, random mask, and unaltered feature baselines have shorter obfuscation duration, but suffer from either substantial utility degradation or greater reconstruction-based privacy leakage, as evidenced in Tables I and II. The PCA embedding method has the longest obfuscation duration due to the computational cost of singular value decomposition.

Importantly, this overhead is modest relative to the substantial gains in utility and privacy achieved by LLPRI, as shown in Tables I and II. In the single-server setting, the total duration of LLPRI is approximately 48 ms as reported in Table III(a). This indicates that LLPRI reduces privacy leakage while maintaining end-to-end inference latency within a practical range for latency-sensitive applications.

Furthermore, comparing the multi-server configuration of LLPRI in Table III(c) with its single-server counterpart in Table III(a), we observe that the multi-server configuration achieves lower latency than the single-server configuration. For example, the detection latency decreases from $7.530 + 40.978$ ms to $7.560 + 32.721$ ms. This result shows that the scheduler can exploit parallel computation across multiple edge servers to reduce server-side inference time. It also demonstrates the practicality of LLPRI in distributed Edge-IoT environments where multiple edge servers are available.

TABLE III
END-TO-END TASK DURATION OF ALL SCHEMES

Obfuscator	Obfuscation System (local)	Target model (server)	
		Detection	Segmentation
LLPRI	7.530 ms	40.978 ms	42.621 ms
Unaltered Feature	2.007 ms	41.293 ms	43.185 ms
Gaussian Noise	4.915 ms	41.994 ms	43.611 ms
Random Mask	1.263 ms	40.214 ms	42.241 ms
PCA Embedding	224.728 ms	43.828 ms	43.826 ms

(a) Overall task duration on the local device and edge server in the single-server setting.

Sensitive object detector	Scheduler	Obfuscator
5.2709 ms	2.9376×10^{-2} ms	2.2600 ms

(b) Duration for each module in LLPRI.

Obfuscation System (local)	Target model (server)	
	Detection	Segmentation
7.560 ms	32.721 ms	33.179 ms

(c) Overall task duration of LLPRI in the multi-server scenario.

VI. Ablation Studies

In this section, we conduct two ablation studies to examine key design choices that strongly affect the end-to-end latency, reconstruction-based privacy leakage, and utility of LLPRI. First, we evaluate alternative architectures for the sensitive

object detector. Its accuracy directly affects the correctness of sensitive-object identification and, consequently, the level of privacy protection, while its runtime contributes substantially to the overall system overhead (see Table III(b)). Second, we study the effect of selecting different split points within the backbone network, where the model is divided into the client embedding network and the server network (see Sec. IV-C). The choice of split point determines how much computation remains on the local device and how much structural detail is preserved in the transmitted features, thereby shaping the privacy–utility–latency tradeoff. These two design choices are fundamental to the behavior of LLPRI, and understanding their impact is essential for configuring the system to meet different deployment requirements.

A. Comparison of Sensitive Object Detector Variants

TABLE IV
COMPARISON OF SENSITIVE OBJECT DETECTORS

Detector	mAP50	SSIM	SIM		NMI	Runtime (ms)
			ResNet50	VGG16		
LLPRI's Detector	0.6036	0.1490	0.4973	0.1269	0.1010	5.27
Faster-RCNN	0.6301	0.0955	0.4795	0.1120	0.0670	17.31
SSD	0.3940	0.2305	0.5435	0.1822	0.1412	10.76

Note: mAP50 is used as the detection accuracy metric.

In this subsection, we evaluate the effectiveness of the YOLOv8n-based detector adopted by LLPRI (see Sec. IV-A) by comparing it with two representative object detection frameworks: 1) Faster R-CNN, a widely used high-performance two-stage detector; and 2) SSD [56], a lightweight one-stage model designed for efficient inference.

As shown in Table IV, the three detectors exhibit distinct speed–accuracy tradeoffs. Faster R-CNN achieves the highest accuracy and, accordingly, the lowest privacy leakage, as indicated by its lowest SSIM, SIM, and NMI values. However, it is also the slowest detector: its two-stage region proposal pipeline results in a runtime nearly $3\times$ longer than that of LLPRI's detector, making it less suitable for the latency-sensitive settings targeted by LLPRI.

Although SSD is lighter than Faster R-CNN, it remains roughly $2\times$ slower than LLPRI's detector while also delivering the lowest accuracy among the three models. This reduced accuracy can lead to missed or poorly localized sensitive objects, resulting in higher privacy leakage. Therefore, SSD is less effective in scenarios that require both low latency and reliable sensitive-object detection.

In contrast, LLPRI's detector offers the most favorable overall balance. It achieves the lowest latency while maintaining detection accuracy close to that of Faster R-CNN. This competitive accuracy translates into strong privacy performance, substantially better than that of SSD and only slightly below that of Faster R-CNN. At the same time, its low runtime reduces the overhead of local privacy-preserving processing, making it suitable for latency-sensitive visual inference.

Finally, although LLPRI currently adopts a YOLOv8n-based detector, the system is not tied to any specific detection architecture. Detectors with improved accuracy–latency tradeoffs or better deployment support can be integrated without modifying the scheduler or obfuscator. Moreover, the enhancements introduced in Sec. IV-A, namely the privacy-aware losses and knowledge distillation, can be applied to a new detector with minimal adjustment. This modularity allows LLPRI to benefit from future advances in object detection.

B. Impact of Split Point on Performance

TABLE V
PERFORMANCE COMPARISON ACROSS DIFFERENT SPLIT POINTS

Split Layer	Utility degradation (mAP50)		SSIM	Runtime (Local Device)
	Detection	Segmentation		
Residual Layer 1	14.99%	14.32%	0.1490	2.2600 ms
Residual Layer 2	16.37%	16.01%	0.1377	2.7959 ms
Residual Layer 3	17.42%	17.08%	0.1328	3.5702 ms

In this subsection, we evaluate how the split point affects the privacy–utility–latency tradeoff of LLPRI. Following the same experimental setup as before, we adopt ResNet-18 as the backbone and place the split at different depths. Specifically, we consider three candidate split locations: Residual Layer 1, Residual Layer 2, and Residual Layer 3. We use SSIM as the primary indicator of reconstruction-based privacy leakage in this ablation because it directly measures reconstruction similarity and is aligned with the privacy loss used during obfuscator training.

As shown in Table V, Residual Layer 1 requires the least on-device computation and therefore achieves the shortest local runtime. However, its activations preserve more spatial and textural details, leading to higher SSIM values and consequently greater privacy leakage. In contrast, Residual Layer 3 shifts more computation to the local device, resulting in a longer runtime. However, its deeper activations are more abstract, which reduces privacy leakage, as reflected by the lower SSIM values. This increased abstraction also results in slightly greater utility degradation because less low-level information is retained in the transmitted features. Despite these differences, all three split locations achieve competitive privacy and utility performance, indicating that the learned masking module remains effective across different split points.

In addition, the relatively modest differences observed across the split points may be attributable to the shallow depth of the ResNet-18 backbone; between Residual Layer 1 and Residual Layer 3, the depth does not change dramatically. Nevertheless, the results still demonstrate a consistent privacy–utility–latency tradeoff across the split locations.

Overall, earlier split points favor latency and utility, whereas later split points tend to favor privacy. Users can select the most suitable split point based on their latency constraints, privacy requirements, and utility expectations.

VII. DISCUSSION

System computational overhead. The majority of the computational overhead in LLPRI comes from the sensitive object detector. However, as shown in Sec. V-F, the overall system can still maintain low end-to-end latency in the multi-server setting. For video applications with more stringent latency

requirements, the detector does not necessarily need to run on every frame. Instead, detection can be performed periodically, while object-tracking methods are used to identify sensitive objects between detection intervals. This strategy can further reduce detector overhead and improve the practicality of LLPRI for latency-sensitive visual IoT applications.

System storage overhead. In LLPRI, the YOLOv8n-based detector contains ~ 3.2 million parameters, while the ResNet-18-based obfuscator contains $\sim$ 11.7 million parameters. Using the standard 32-bit floating-point format for parameter storage, the two models require only ~ 59.6 MB of storage. This requirement is relatively small compared with the storage capacity of modern visual IoT devices [57].

Distributed deployment and scheduling. LLPRI is designed for distributed Edge-IoT environments in which multiple edge servers can support collaborative inference. Its latency benefits mainly arise from exploiting parallel computation and available resources across servers, rather than relying on a single high-performance node. The scheduler is modular and can be replaced or extended with alternative scheduling policies based on deployment constraints. Although the current design does not directly optimize communication overhead, communication-aware scheduling, bandwidth-aware partitioning, and adaptive compression could be incorporated in future work to further improve deployment efficiency in bandwidth-constrained IoT environments.

Generalization to complex vision models. Our experiments demonstrate the effectiveness of LLPRI on CNN-based detection and segmentation models. The modular design of LLPRI can also support more complex vision models because the sensitive object detector and scheduler operate independently of the target model architecture. The obfuscator is the only component directly tied to the target model, and its integration mainly requires selecting a split point and training a mask generation network for the corresponding intermediate features. For transformer-based architectures such as DETR [58], LLPRI can be applied by selecting an appropriate split point within the encoder backbone. Moreover, LLPRI is not restricted to models with conventional backbones. For architectures such as Segmenter [59], the embedding or tokenization module can be placed on the local device, and the mask generation network can obfuscate privacy-sensitive information in the resulting embeddings.

Empirical privacy evaluation. LLPRI targets the practical privacy–utility–latency tradeoff in low-latency visual inference. Formal privacy guarantees, such as those provided by HE or DP, are difficult to apply directly in this setting without introducing substantial computational overhead or utility degradation. Therefore, we evaluate privacy leakage empirically against reconstruction-based adversaries and complement perceptual and semantic similarity metrics with an information-theoretic metric, NMI. MI provides a principled way to quantify the statistical dependence between the original input and the adversary's reconstruction and thus characterizes how much information about the original visual data remains recoverable after obfuscation. Together, these metrics provide meaningful evidence of resistance to reconstruction attacks under the considered threat model.

Private training. LLPRI focuses on privacy-preserving inference after deployment. We assume that the client embedding network and mask generation network are trained in a trusted environment before deployment. Privacy-preserving training of these components is an orthogonal problem and is outside the scope of this work. Future work may integrate LLPRI with privacy-preserving training approaches, such as federated learning combined with differential privacy or homomorphic encryption, as well as privacy-aware knowledge distillation, to further reduce privacy leakage across the model training and inference stages; see [26], [27], [60] and references therein.

VIII. Conclusion

We propose LLPRI, a low-latency privacy-preserving vision inference system that balances privacy and utility. LLPRI distinguishes between sensitive and non-sensitive objects, maximizing utility for the latter while balancing privacy and utility for the former. It also incorporates a scheduler that efficiently leverages computational resources in multi-server distributed environments. Experiments show that LLPRI effectively preserves user privacy with minimal utility loss and low latency.

References

[1] J. Aggarwal and M. Ryoo, "Human activity analysis: A review," *ACM Comput. Surv.*, vol. 43, no. 3, pp. 1–43, 4 2011.

[2] M. Speicher, B. D. Hall, and M. Nebeling, "What is mixed reality?" in *Proc. CHI Conf. Hum. Factors Comput. Syst.*, 2019, pp. 1–15.

[3] Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, "Object detection in 20 years: A survey," *Proc. IEEE*, vol. 111, no. 3, pp. 257–276, 2023.

[4] S. Minaee, Y. Boykov, F. Porikli, A. Plaza, N. Kehtarnavaz, and D. Terzopoulos, "Image segmentation using deep learning: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 7, pp. 3523–3542, 2022.

[5] Amazon Web Services. (2024) Free cloud computing services - AWS free tier. [Online]. Available: https://aws.amazon.com/

[6] K. Du *et al.*, "Server-driven video streaming for deep learning inference," in *Proc. ACM SIGCOMM Conf.*, 2020, pp. 557–570.

[7] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, 2016.

[8] W. Zhang *et al.*, "Elf: Accelerate high-resolution mobile deep vision with content-aware parallel offloading," in *Proc. ACM Int. Conf. Mobile Comput. Netw. (MobiCom)*, 2021, pp. 201–214.

[9] J. Ma, B. Shang, H. Song, Y. Huang, and P. Fan, "Reliability versus latency in iiot visual applications: A scalable task offloading framework," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 16 726–16 735, 2022.

[10] Y. Sahni, J. Cao, L. Yang, and S. Wang, "Distributed resource scheduling in edge computing: Problems, solutions, and opportunities," *Comput. Netw.*, vol. 219, p. 109430, 2022.

[11] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, "(Leveled) fully homomorphic encryption without bootstrapping," *ACM Trans. Comput. Theory*, vol. 6, no. 3, pp. 1–36, 7 2014.

[12] W. Du and M. J. Atallah, "Secure multi-party computation problems and their applications: A review and open problems," in *Proc. New Security Paradigms Workshop (NSPW)*, 2001, pp. 13–22.

[13] C. Dwork, "Differential privacy: A survey of results," in *Proc. Theory Appl. Models Comput. (TAMC)*, 2008, pp. 1–19.

[14] Y. Matsubara, M. Levorato, and F. Restuccia, "Split computing and early exiting for deep learning applications: Survey and research challenges," *ACM Comput. Surv.*, vol. 55, no. 5, pp. 1–30, 12 2022.

[15] S. A. Khowaja, A. Waqas, M. Tabrez Quasim, and K. Dev, "Post: Pruning oriented security for inversion attack in edge-based internet of things," *IEEE Internet Things J.*, vol. 13, no. 3, pp. 4411–4420, 2026.

[16] H. Zhao, J. Chi, Y. Tian, and G. J. Gordon, "Trade-offs and guarantees of adversarial representation learning for information obfuscation," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9485–9496.

[17] Y. Hu, M. Zhu, Q. Jin, F. Qian, and B. Li, "Magiccloth: Protect user privacy in AR streaming," in *Proc. ACM Workshop Mobile Immersive Comput., Netw., Syst. (ImmerCom)*, 2023, pp. 222–228.

[18] F. Qian and B. Li, "Boosting remote multi-user AR privacy through a magic rope," in *Proc. ACM Int. Conf. Mobile Syst. Appl. Services (MobiSys)*, 2022, pp. 583–584.
[19] A. Li, J. Guo, H. Yang, and Y. Chen, "Deepobfuscator: Adversarial training framework for privacy-preserving image classification," *arXiv preprint arXiv:1909.04126*, vol. 2, no. 3, 2019.
[20] A. Singh *et al.*, "Disco: Dynamic and invariant sensitive channel obfuscation for deep neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 12 125–12 135.
[21] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "CryptoNets: Applying neural networks to encrypted data with high throughput and accuracy," in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 48, 6 2016, pp. 201–210.
[22] J. Liu, M. Juuti, Y. Lu, and N. Asokan, "Oblivious neural network predictions via minionn transformations," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, 2017, pp. 619–631.
[23] G. He and Z. Yuan, "Verifiable privacy-preserving neural network inference via multi-key homomorphic encryption," *IEEE Internet Things J.*, vol. 13, no. 10, pp. 22 072–22 081, 2026.
[24] A. Brutzkus, R. Gilad-Bachrach, and O. Elisha, "Low latency privacy preserving inference," in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 97, 6 2019, pp. 812–821.
[25] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan, "GAZELLE: A low latency framework for secure neural network inference," in *Proc. USENIX Security Symp. (USENIX Security)*, 8 2018, pp. 1651–1669.
[26] M. Abadi *et al.*, "Deep learning with differential privacy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, 2016, pp. 308–318.
[27] S. Truex, L. Liu, K. Chow, M. E. Gursoy, and W. Wei, "LDP-Fed: Federated learning with local differential privacy," in *Proc. ACM Int. Workshop Edge Syst. Analytics Netw. (EdgeSys)*, 2020, pp. 61–66.
[28] Y. Zhao, J. Zhao, M. Yang, T. Wang, N. Wang, L. Lyu, D. Niyato, and K.-Y. Lam, "Local differential privacy-based federated learning for internet of things," *IEEE Internet Things J.*, vol. 8, no. 11, pp. 8836–8853, 2021.
[29] F. Mireshghallah, M. Taram, P. Ramrakhyani, A. Jalali, D. Tullsen, and H. Esmaeilzadeh, "Shredder: Learning noise distributions to protect inference privacy," in *Proc. Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, 2020, pp. 3–18.
[30] Z. He, T. Zhang, and R. B. Lee, "Attacking and protecting data privacy in edge–cloud collaborative inference systems," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9706–9716, 2021.
[31] S. A. Osia *et al.*, "A hybrid deep learning architecture for privacy-preserving mobile analytics," *IEEE Internet Things J.*, vol. 7, no. 5, pp. 4505–4518, 2020.
[32] J. Fang, X. Wang, Y. Liu, H. Tang, and X. Li, "Multiagent collaborative inference optimization for large-scale dnns in iot edge systems," *IEEE Internet Things J.*, vol. 13, no. 11, pp. 24 938–24 953, 2026.
[33] X. Gitiaux and H. Rangwala, "Fair representations by compression," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, vol. 35, no. 13, 5 2021, pp. 11 506–11 515.
[34] A. Li, Y. Duan, H. Yang, Y. Chen, and J. Yang, "TIPRDC: Task-independent privacy-respecting data crowdsourcing framework for deep learning with anonymized intermediate representations," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min. (KDD)*, 2020, pp. 824–832.
[35] P. C. Roy and V. N. Boddeti, "Mitigating information leakage in image representations: A maximum entropy approach," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 6 2019.
[36] D. Xu, M. Zheng, L. Jiang, C. Gu, R. Tan, and P. Cheng, "Lightweight and unobtrusive data obfuscation at iot edge for remote inference," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9540–9551, 2020.
[37] M. Corbett, B. David-John, J. Shang, Y. C. Hu, and B. Ji, "BystandAR: Protecting bystander visual data in augmented reality systems," in *Proc. ACM Int. Conf. Mobile Syst. Appl. Services (MobiSys)*, 2023, pp. 370–382.
[38] A. Horé and D. Ziou, "Image quality metrics: PSNR vs. SSIM," in *2010 20th Int. Conf. Pattern Recognit.*, 2010, pp. 2366–2369.
[39] U. Sara, M. Akter, and M. S. Uddin, "Image quality assessment through FSIM, SSIM, MSE and PSNR—a comparative study," *J. Comput. Commun.*, vol. 7, no. 3, pp. 8–18, 2019.
[40] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.
[41] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.
[42] T. M. Cover, *Elements of information theory*. John Wiley & Sons, 1999.
[43] D. B. Russakoff, C. Tomasi, T. Rohlfing, and C. R. Maurer, "Image similarity using mutual information of regions," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2004, pp. 596–607.
[44] T.-Y. Lin *et al.*, "Microsoft coco: Common objects in context," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2014, pp. 740–755.
[45] G. Jocher, A. Chaurasia, and J. Qiu. (2023) Ultralytics YOLOv8. [Online]. Available: https://github.com/ultralytics/ultralytics
[46] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge distillation: A survey," *Int. J. Comput. Vis.*, vol. 129, no. 6, pp. 1789–1819, 6 2021.
[47] C. Song and V. Shmatikov, "Overlearning reveals sensitive attributes," *arXiv preprint arXiv:1905.11742*, 2019.
[48] T. Bai, J. Luo, J. Zhao, B. Wen, and Q. Wang, "Recent advances in adversarial training for adversarial robustness," *arXiv preprint arXiv:2102.01356*, 2021.
[49] A. M. Hafiz and G. M. Bhat, "A survey on instance segmentation: state of the art," *Int. J. Multimedia Inf. Retrieval*, vol. 9, no. 3, pp. 171–189, 2020.
[50] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 28, 2015.
[51] K. He, G. Gkioxari, P. Dollar, and R. Girshick, "Mask R-CNN," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 10 2017.
[52] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Proc. Int. Conf. Med. Image Comput. Comput.-Assist. Intervent. (MICCAI)*, 2015, pp. 234–241.
[53] P. K. Mall *et al.*, "A comprehensive review of deep neural networks for medical image processing: Recent developments and future opportunities," *Healthcare Analytics*, vol. 4, p. 100216, 2023.
[54] A. Rea and W. Rea, "How many components should be retained from a multivariate time series pca?" *arXiv preprint arXiv:1610.03588*, 2016.
[55] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Foundations and Trends® in Theoretical Computer Science*, vol. 9, no. 3–4, pp. 211–407, 2014.
[56] W. Liu *et al.*, "Ssd: Single shot multibox detector," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2016, pp. 21–37.
[57] NVIDIA. (2026) Getting started with Jetson Nano developer kit. [Online]. Available: https://developer.nvidia.com/embedded/learn/get-started-jetson-nano-devkit
[58] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020, pp. 213–229.
[59] R. Strudel, R. Garcia, I. Laptev, and C. Schmid, "Segmenter: Transformer for semantic segmentation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 10 2021, pp. 7262–7272.
[60] J. Zhang, R. Sequeira, and K. Psounis, "Spinml: Customized synthetic data generation for private training of specialized ml models," *Proc. Priv. Enhanc. Technol.*, 2025.



Te-Yi Kan received the B.S. degree in Electrical Engineering from National Taiwan University, Taiwan, in 2019, and the M.S. degree in Electrical and Computer Engineering from the University of California, Los Angeles, CA, USA, in 2020. From 2020 to 2022, he was a Research Assistant with the Academia Sinica, Taiwan. He is currently a Ph.D. student in Electrical and Computer Engineering at the University of Southern California. His research interests include wireless communications, wireless networks, and privacy-enhanced machine learning.



Konstantinos Psounis received the BS degree from the Department of Electrical and Computer Engineering, National Technical University of Athens, Greece, in 1997, and the M.S. and Ph.D. degrees in Electrical Engineering from Stanford, CA, USA, in 1999 and 2002, respectively. He is a Professor of Electrical and Computer Engineering and of Computer Science with the University of Southern California. He is an IEEE Fellow and a Distinguished Member of the ACM.